

Aprobación Firmware V5.1

Usuario: [File: PROMPT_AUDITORIA_RONDA8_ASTRAPROBACION.md]

use.ai: [RECHAZADO] como aprobación final del firmware V5.1 para producción y como autorización incondicional de ensayos hidráulicos. La revisión estática muestra mejoras reales, pero no demuestra todavía integridad metrológica completa, seguridad hidráulica ni funcionamiento del firmware integrado.

Este dictamen **no rechaza investigar experimentalmente el YF-S401** por debajo de su rango nominal. Esa caracterización es legítima si se identifica como tal, se conserva el dato primario y se realiza en un banco supervisado con protección hidráulica independiente.

Mi alcance es una revisión técnica documental. No he compilado ni ejecutado el código recibido o el propuesto aquí, ni he observado el montaje. Por tanto, no sería sincero presentar esta respuesta como una certificación inapelable o afirmar que un código nuevo ya está validado para producción.

1. Hallazgos que impiden la aprobación final

1.1. El expediente no contiene el firmware completo

Se proporcionan configuración, adquisición de caudal y registro, pero faltan **los componentes que determinan la integración y la seguridad**:

- El programa principal, las tareas y su sincronización.

- El generador de pasos, las rampas y la secuencia de parada.

- El servidor, DNS del portal cautivo y exportación de datos.

- La implementación del balance y del estado de Darcy.

- El tratamiento de fallos de inicialización, desbordamiento del registro y pérdida de alimentación.

Los valores de compilación son **resultados declarados por ustedes**, no resultados reproducidos en esta revisión. Tampoco permiten verificar el comportamiento temporal.

La ausencia de esos componentes no prueba que estén mal, pero sí impide aprobarlos. En particular, no puedo confirmar la afirmación de apertura automática del portal en cualquier dispositivo a partir de un código que no lo incluye.

1.2. El modo seco no aísla el estimador de caudal

En `setModoSeco()` se borran `_candidato` y `_altoConocido`, pero se conservan:

```
_hayUltimo .  
_ultimoPulso_us .  
_ultimoPeriodo_us .
```

Los períodos ya acumulados en la ventana.

Además, en modo seco la ISR sigue actualizando los períodos. En consecuencia, **el ruido aceptado en seco puede generar caudal y contaminar la estimación al regresar a modo húmedo.**

Excluir esos pulsos del volumen es correcto, pero insuficiente. Las transiciones necesitan separar las épocas de adquisición, invalidar el período y no mezclar acumuladores temporales de estados distintos.

También debe cambiarse la terminología: un pulso con forma digital admisible no es un pulso **físicamente demostrado como líquido**. El software no distingue una señal Hall auténtica de una interferencia que reproduzca la misma forma temporal.

1.3. La validez de calibración no está vinculada al rango

`calibrado()` devuelve una bandera global; `fueraDeRango()` solo comprueba el máximo y lo hace sobre el caudal filtrado.

Así, la alimentación puede indicar simultáneamente:

```
CAL_A = 1  
FUERA_RANGO_A = 0
```

aunque esté midiendo por debajo de los **200 mL/min** del intervalo de calibración declarado. Tampoco coincide el máximo programado de 1400 mL/min con el máximo declarado de 1360 mL/min.

Se deben distinguir **calibración documentada**, **período disponible** y **medición dentro del intervalo calibrado**. Son propiedades diferentes.

No objeciono mantener el coeficiente provisional del permeado. Objeciono presentar como validada una medición para la que solo existe un coeficiente supuesto.

1.4. El registro no garantiza fronteras de sesión coherentes

`RegistroEnsayos` recibe contadores de la última llamada a `actualizar()`, junto con una marca temporal externa que **no está vinculada por el propio módulo a esos contadores**.

Por ejemplo:

Se actualiza el sensor.

Llegan pulsos.

Se inicia una sesión usando los acumulados anteriores.

La siguiente actualización incorpora los pulsos pendientes anteriores al inicio.

La diferencia entre el contador inicial y el final puede entonces atribuir a la sesión pulsos previos. El problema inverso aparece al finalizar sin una adquisición final.

Además, `iniciar()` **no rechaza una sesión ya activa**. Una segunda llamada cambia el identificador y el origen temporal, dejando la sesión anterior sin `FIN`.

La solución requiere capturar contadores y tiempo **en la misma operación protegida**, registrar una instantánea inicial y otra final, y definir explícitamente cómo se asigna el pulso que cruza la frontera.

1.5. No está demostrada la protección de presión

Las constantes `TMP_MAX_SEGURA_BAR` y `P1_MAX_SEGURA_BAR` **no implementan protección alguna** en el código mostrado.

Sin instrumentación de presión, el firmware no puede garantizar el cumplimiento de TMP. Tampoco puede inferirse presión segura a partir de RPM o caudal nominal.

Antes de ensayar con la membrana debe existir **una protección hidráulica independiente y verificada**, con supervisión de presión apropiada al circuito, alivio o limitación efectiva y parada accesible. La configuración concreta debe validarla el responsable técnico del banco.

No es necesario esperar al Subhito 2.3 para caracterizar sensores en un circuito de descarga libre, pero eso no equivale a autorizar cualquier ensayo de ultrafiltración.

2. Evaluación de la arquitectura

La arquitectura es **adecuada como punto de partida, no una solución de máximo rigor ya demostrada**.

La continuidad temporal de 64 bits, la formulación recíproca y el totalizador independiente son decisiones correctas. Espressif documenta `esp_timer_get_time()` como una función rápida utilizable tanto en tareas como en ISR; eso respalda su elección, aunque no elimina la latencia del servicio de interrupciones. [Espressif](#)

2.1. Qué puede y qué no puede resolver el filtro digital

Con los valores informados:

$$\tau = RC = 4700 \times 200 \times 10^{-9} = 940 \mu s$$

Para alimentación a 1360 mL/min, usando el coeficiente declarado:

$$f = 196.50 \times 1.36 = 267.24 \text{ Hz}$$

$$T \approx 3742 \mu s$$

Un semiperíodo del 50 % sería aproximadamente 1871 μs , cercano a dos constantes de tiempo. **El acondicionamiento analógico no es despreciable.**

Ese cálculo no demuestra que el circuito falle: faltan topología completa, salida del sensor, ciclo de trabajo y umbrales reales del GPIO. Sí demuestra que deben medirse las formas de onda en el pin, no dar por garantizada su fidelidad.

El filtro por anchos:

Rechaza determinadas perturbaciones cortas.

No recupera pulsos eliminados antes del GPIO.

No distingue interferencia y señal cuando ambas cumplen el mismo patrón.

Puede sesgar el resultado si sus umbrales eliminan pulsos verdaderos.

No evita la carga de CPU causada por cada interrupción de una ráfaga EMI.

Los umbrales deben validarse con osciloscopio y trenes de pulsos conocidos, tanto con el motor detenido como funcionando.

2.2. Caudal recíproco y caudal medio no son intercambiables

En flujo pulsátil, **el estimador recíproco es útil para indicación**, pero el caudal medio de un intervalo experimental debe derivarse preferentemente de los incrementos de conteo:

$$\overline{Q}_{\mathrm{mL/min}}$$

$$\frac{\Delta N \times 10^9}{K \Delta t_{\mu s}}$$

El volumen correspondiente, si el modelo de coeficiente constante resulta válido, es:

$$\Delta V_{\mathrm{L}} = \frac{\Delta N}{60K}$$

Un filtro exponencial fijo de 0.25 cambia su respuesta temporal si cambia la frecuencia de `actualizar()`. Por eso, la propuesta siguiente **registra caudal recíproco sin suavizar** y deja cualquier filtro cosmético fuera del dato primario.

2.3. Capacidad real del registro

Con una sola sesión, 600 filas permiten **1 inicio, 598 periódicas y 1 final**. No equivalen a 600 muestras periódicas más sus eventos.

Con 20 sesiones, los eventos consumen 40 filas y quedan, como máximo, **560 filas periódicas compartidas**. La memoria RAM tampoco proporciona persistencia ante reinicio.

3. Implementación correctiva completa del núcleo metrológico

Entrego una sustitución completa de adquisición y registro, con un programa de comprobación e integración mínima. **No la presento como firmware integral de producción validado**: faltan los módulos del proyecto no entregados y las pruebas físicas.

Esta versión introduce deliberadamente una API nueva basada en **instantáneas inmutables**. No debe reemplazarse a ciegas conservando las llamadas antiguas del SCADA.

Sus decisiones principales son:

Conteos acumulados de 64 bits, sin reinicios entre sesiones.

Tiempo y conteos capturados juntos para cada sensor.

Modo seco separado, sin caudal ni volumen estimados a partir de sus pulsos.

Épocas de configuración, para detectar cambios durante una sesión.

Caudales no disponibles como NAN, no ceros sintéticos.

Reserva de espacio para FIN y rechazo de inicios superpuestos.

Tiempos individuales por sensor, sin afirmar simultaneidad inexistente.

Exportación del dato primario, la calidad y las omisiones.

El volumen se denomina *estimado*: corresponde a pulsos aceptados en modo de medición, no a una prueba de presencia de agua.

3.1. Archivo `config_metrologia.h`

Este archivo es independiente del `config.h` original. Las declaraciones de calibración quedan desactivadas hasta incorporar evidencia revisada; eso no impide la caracterización.

```
#pragma once

#include <Arduino.h>
#include <stddef.h>
#include <stdint.h>

namespace Metrologia {

// Pines del montaje informado. Este modulo usa solo los dos sensores.
constexpr uint8_t PIN_PUL = 18;
constexpr uint8_t PIN_DIR = 19;
constexpr uint8_t PIN_ENA = 23;
constexpr uint8_t PIN_LED = 2;
constexpr uint8_t PIN_A = 14;
constexpr uint8_t PIN_P = 27;
constexpr uint8_t PIN_SDA = 21;
constexpr uint8_t PIN_SCL = 22;

// Verificacion de todos los pares, no solo sensor contra motor.
constexpr uint8_t PINES[] = {
    PIN_PUL, PIN_DIR, PIN_ENA, PIN_LED,
    PIN_A, PIN_P, PIN_SDA, PIN_SCL
};

constexpr bool pinesUnicos() {
    for (size_t i = 0; i < sizeof(PINES) / sizeof(PINES[0]); ++i) {
        for (size_t j = i + 1;
            j < sizeof(PINES) / sizeof(PINES[0]); ++j) {
            if (PINES[i] == PINES[j]) return false;
        }
    }
    return true;
}
```

```

}

static_assert(pinesUnicos(), "Hay GPIO compartidos");

// Los umbrales son candidatos a validar en banco.
// No constituyen por si mismos una caracterizacion del sensor.
struct ConfigSensor {
    uint8_t pin;
    double k_Hz_por_Lmin;

    uint32_t lowMin_us;
    uint32_t highMin_us;
    uint32_t periodoMin_us;
    uint32_t timeout_us;

    double qMaxOperativo_mLmin;

    bool calibracionDocumentada;
    double qMinCal_mLmin;
    double qMaxCal_mLmin;
};

constexpr ConfigSensor SENSOR_A = {
    PIN_A, 196.50,
    200, 200, 1000, 1000000,
    1400.0,
    false, 200.0, 1360.0
};

constexpr ConfigSensor SENSOR_P = {
    PIN_P, 687.33,
    600, 600, 2500, 5000000,
    200.0,
    false, 0.0, 0.0
};

constexpr size_t MAX_FILAS = 600;
constexpr uint16_t MAX_SESIONES = 20;
constexpr int64_t INTERVALO_LOG_US = 10000000;

} // namespace Metrologia

```

3.2. Archivo caudalimetro.h

La concurrencia tiene un contrato explícito: **una tarea propietaria** llama a `capturar()`, `setModoSeco()` y `limpiarAlarmaSeco()`. Las otras tareas reciben copias mediante una cola o un mecanismo de publicación sincronizado.

```

#pragma once

#include <Arduino.h>
#include <stdint.h>
#include "config_metrologia.h"

class Caudalimetro {
public:

```

```

enum Calidad : uint16_t {
    INICIADO                = 1U << 0,
    CAL_DOCUMENTADA         = 1U << 1,
    PERIODO_DISPONIBLE     = 1U << 2,
    SIN_SENÁL              = 1U << 3,
    FUERA_RANGO_CAL        = 1U << 4,
    FUERA_RANGO_OPERATIVO  = 1U << 5,
    SECO                   = 1U << 6,
    RUIDO_SECO             = 1U << 7,
    SIN_IMPULSION          = 1U << 8,
    MEDICION_EN_RANGO_CAL  = 1U << 9
};

// Una instantanea es un valor independiente del objeto sensor.
// t_us corresponde al instante de captura de sus contadores.
struct Muestra {
    int64_t t_us = 0;
    int64_t ultimoAceptado_us = -1;
    int64_t edadPulso_us = -1;

    uint64_t flancos = 0;
    uint64_t aceptados = 0;
    uint64_t pulsosMedicion = 0;
    uint64_t rechazados = 0;

    uint32_t epoca = 0;
    uint16_t calidad = 0;

    double k = 0.0;
    double frecuencia_Hz = 0.0;
    double q_mLmin = 0.0;
    double volumenEstimado_L = 0.0;
};

explicit Caudalimetro(const Metrologia::ConfigSensor& config);

Caudalimetro(const Caudalimetro&) = delete;
Caudalimetro& operator=(const Caudalimetro&) = delete;

// Requiere gpio_install_isr_service(0) antes de llamar.
// Los objetos deben vivir durante toda la adquisicion.
bool begin();

// Solo la tarea propietaria llama a estos metodos.
Muestra capturar(bool bombaEmpuja);

// Cambiar de modo inicia una nueva epoca y borra la historia
// del estimador, pero conserva todos los contadores acumulados.
void setModoSeco(bool activo);
void limpiarAlarmaSeco();

// Primitiva pura para analisis de dos instantaneas.
// Devuelve NAN si el intervalo o su configuracion no son validos.
static double caudalMedio_mLmin(
    const Muestra& inicial, const Muestra& final);

private:
    static void isrPunto(void* arg);

```

```

static void ISRISR(void arg),
void isr();

// Llamar solo dentro de la seccion critica.
void reiniciarEstimadorLocked();

const Metrologia::ConfigSensor _cfg;
portMUX_TYPE _mux = portMUX_INITIALIZER_UNLOCKED;

bool _iniciado = false;

// Todas las variables siguientes se comparten con la ISR
// exclusivamente bajo _mux.
bool _seco = false;
bool _alarmaSeco = false;
bool _nivelAlto = false;
bool _altoConocido = false;
bool _candidato = false;
bool _hayAnterior = false;

int64_t _inicioNivel_us = 0;
int64_t _bajada_us = 0;
int64_t _bajadaAnterior_us = 0;
int64_t _ultimoAceptado_us = -1;
int64_t _ultimoPeriodo_us = 0;

uint64_t _flancos = 0;
uint64_t _aceptados = 0;
uint64_t _pulsosMedicion = 0;
uint64_t _rechazados = 0;

uint64_t _periodosVentana = 0;
uint64_t _sumaPeriodosVentana_us = 0;

uint32_t _epoca = 1;

// Privado de la tarea propietaria.
uint64_t _anteriorMedicion = 0;
};

```

3.3. Archivo `caudalimetro.cpp`

Se utiliza el servicio GPIO de ESP-IDF **sin declarar seguridad IRAM**. Esto evita afirmar una propiedad no auditada de toda la cadena de ejecución. Su consecuencia es explícita: pueden perderse flancos si el servicio queda diferido durante operaciones de flash; esas operaciones deben excluirse durante el ensayo o debe validarse una implementación IRAM completa.

Espressif distingue precisamente entre interrupciones ordinarias y manejadores cuyo código y dependencias deben residir en memoria interna cuando se usan las opciones IRAM. [Espressif](#)

```

#include "caudalimetro.h"

#include <math.h>
#include "driver/gpio.h"
#include "esp_timer.h"

```

```

Caudalimetro::Caudalimetro(
    const Metrologia::ConfigSensor& config)
    : _cfg(config) {}

void Caudalimetro::reiniciarEstimadorLocked() {
    _altoConocido = false;
    _candidato = false;
    _hayAnterior = false;

    _bajada_us = 0;
    _bajadaAnterior_us = 0;
    _ultimoAceptado_us = -1;
    _ultimoPeriodo_us = 0;

    _periodosVentana = 0;
    _sumaPeriodosVentana_us = 0;
}

bool Caudalimetro::begin() {
    if (_iniciado) return false;

    if (!GPIO_IS_VALID_GPIO(_cfg.pin) ||
        !isfinite(_cfg.k_Hz_por_Lmin) ||
        _cfg.k_Hz_por_Lmin <= 0.0 ||
        _cfg.lowMin_us == 0 ||
        _cfg.highMin_us == 0 ||
        _cfg.periodoMin_us == 0 ||
        _cfg.timeout_us <= _cfg.periodoMin_us ||
        !isfinite(_cfg.qMaxOperativo_mLmin) ||
        _cfg.qMaxOperativo_mLmin <= 0.0) {
        return false;
    }

    if (_cfg.calibracionDocumentada &&
        (!isfinite(_cfg.qMinCal_mLmin) ||
         !isfinite(_cfg.qMaxCal_mLmin) ||
         _cfg.qMinCal_mLmin < 0.0 ||
         _cfg.qMaxCal_mLmin <= _cfg.qMinCal_mLmin)) {
        return false;
    }

    const gpio_num_t pin = static_cast<gpio_num_t>(_cfg.pin);

    gpio_config_t io = {};
    io.pin_bit_mask = 1ULL << _cfg.pin;
    io.mode = GPIO_MODE_INPUT;
    io.pull_up_en = GPIO_PULLUP_DISABLE;
    io.pull_down_en = GPIO_PULLDOWN_DISABLE;
    io.intr_type = GPIO_INTR_DISABLE;

    if (gpio_config(&io) != ESP_OK) return false;

    portENTER_CRITICAL(&_mux);
    _nivelAlto = gpio_get_level(pin) != 0;
    _inicioNivel_us = esp_timer_get_time();
    reiniciarEstimadorLocked();
    portEXIT_CRITICAL(&_mux);
}

```

```

    if (gpio_isr_handler_add(pin, isrPuentes, this) != ESP_OK) {
        return false;
    }

    if (gpio_set_intr_type(pin, GPIO_INTR_ANYEDGE) != ESP_OK ||
        gpio_intr_enable(pin) != ESP_OK) {
        gpio_intr_disable(pin);
        gpio_isr_handler_remove(pin);
        return false;
    }

    _iniciado = true;
    return true;
}

void Caudalimetro::isrPuentes(void* arg) {
    static_cast<Caudalimetro*>(arg)->isr();
}

void Caudalimetro::isr() {
    // Tiempo y nivel se leen bajo el mismo bloqueo que los contadores.
    // Es el tiempo de servicio software, no una captura hardware del
    flanco.
    portENTER_CRITICAL_ISR(&_mux);

    const int64_t ahora = esp_timer_get_time();
    const bool alto =
        gpio_get_level(static_cast<gpio_num_t>(_cfg.pin)) != 0;

    ++_flancos;
    if (_seco) _alarmaSeco = true;

    if (alto == _nivelAlto) {
        // Posible evento diferido o transiciones no observadas.
        // Invalidamos la continuidad para no fabricar un periodo.
        ++_rechazados;
        reiniciarEstimadorLocked();
        _inicioNivel_us = ahora;

        portEXIT_CRITICAL_ISR(&_mux);
        return;
    }

    const int64_t duracion = ahora - _inicioNivel_us;

    if (!alto) {
        // Una bajada abre un candidato si el HIGH anterior fue
        conocido
        // y tuvo duracion suficiente.
        _candidato =
            _altoConocido &&
            duracion >= static_cast<int64_t>(_cfg.highMin_us);

        _bajada_us = ahora;
        if (!_candidato) ++_rechazados;
    } else {
        // Si el nivel es alto y no habia candidato, se abre uno
        // Si el nivel es alto y habia candidato, se actualiza el
        // tiempo de duracion
        _candidato = true;
        _bajada_us = 0;
        _inicioNivel_us = ahora;
        _rechazados = 0;
    }
}

```

```

// La subida permite validar el LOW completo.
if (_candidato &&
    duracion >= static_cast<int64_t>(_cfg.lowMin_us)) {

    const int64_t periodo =
        _bajada_us - _bajadaAnterior_us;

    if (!_hayAnterior ||
        periodo >= static_cast<int64_t>(_cfg.periodoMin_us)) {

        ++_aceptados;
        if (!_seco) ++_pulsosMedicion;

        // Se conserva el periodo entre bajadas aceptadas.
        // Un intervalo demasiado largo inicia otra historia.
        if (_hayAnterior &&
            periodo <= static_cast<int64_t>(_cfg.timeout_us)) {
            _ultimoPeriodo_us = periodo;
            ++_periodosVentana;
            _sumaPeriodosVentana_us +=
                static_cast<uint64_t>(periodo);
        } else {
            _ultimoPeriodo_us = 0;
            _periodosVentana = 0;
            _sumaPeriodosVentana_us = 0;
        }

        _bajadaAnterior_us = _bajada_us;

        // La asignacion de conteo a una sesion se realiza
        // cuando el pulso termina de validarse: en esta
subida.
        _ultimoAceptado_us = ahora;
        _hayAnterior = true;
    } else {
        ++_rechazados;
    }
} else if (_candidato) {
    ++_rechazados;
}

_candidato = false;
_altoConocido = true;
}

_nivelAlto = alto;
_inicioNivel_us = ahora;

portEXIT_CRITICAL_ISR(&_mux);
}

void Caudalimetro::setModoSeco(bool activo) {
    portENTER_CRITICAL(&_mux);

    if (_seco != activo) {
        _seco = activo;
        ++_epoca;
    }
}

```

```

        reiniciarEstimadorLocked();
        _nivelAlto =
            gpio_get_level(static_cast<gpio_num_t>(_cfg.pin)) != 0;
        _inicioNivel_us = esp_timer_get_time();
    }

    portEXIT_CRITICAL(&_mux);
}

void Caudalimetro::limpiarAlarmaSeco() {
    portENTER_CRITICAL(&_mux);
    _alarmaSeco = false;
    portEXIT_CRITICAL(&_mux);
}

Caudalimetro::Muestra Caudalimetro::capturar(bool bombaEmpuja) {
    Muestra m;
    m.k = _cfg.k_Hz_por_Lmin;
    m.frecuencia_Hz = NAN;
    m.q_mLmin = NAN;
    m.volumenEstimado_L = NAN;

    if (!_iniciado) {
        m.t_us = esp_timer_get_time();
        return m;
    }

    bool seco;
    bool alarma;
    bool hayAnterior;
    int64_t periodo;
    uint64_t periodos;
    uint64_t suma;

    portENTER_CRITICAL(&_mux);

    m.t_us = esp_timer_get_time();
    m.ultimoAceptado_us = _ultimoAceptado_us;
    m.flancos = _flancos;
    m.aceptados = _aceptados;
    m.pulsosMedicion = _pulsosMedicion;
    m.rechazados = _rechazados;
    m.epoca = _epoca;

    seco = _seco;
    alarma = _alarmaSeco;
    hayAnterior = _hayAnterior;
    periodo = _ultimoPeriodo_us;

    periodos = _periodosVentana;
    suma = _sumaPeriodosVentana_us;
    _periodosVentana = 0;
    _sumaPeriodosVentana_us = 0;

    portEXIT_CRITICAL(&_mux);

    m.calidad = INICIADO;

```

```

if (_cfg.calibracionDocumentada) {
    m.calidad |= CAL_DOCUMENTADA;
}
if (seco) m.calidad |= SECO;
if (alarma) m.calidad |= RUIDO_SECO;

if (hayAnterior && m.ultimoAceptado_us >= 0) {
    m.edadPulso_us = m.t_us - m.ultimoAceptado_us;
}

const bool sinSenal =
    !hayAnterior ||
    m.edadPulso_us < 0 ||
    m.edadPulso_us > static_cast<int64_t>(_cfg.timeout_us);

if (sinSenal) m.calidad |= SIN_SENAL;

// El modo seco no produce una estimacion de caudal.
if (!seco && !sinSenal && periodo > 0) {
    m.calidad |= PERIODO_DISPONIBLE;

    m.frecuencia_Hz =
        (periodos > 0 && suma > 0)
        ? 1.0e6 * static_cast<double>(periodos) /
          static_cast<double>(suma)
        : 1.0e6 / static_cast<double>(periodo);

    m.q_mLmin = 1000.0 * m.frecuencia_Hz / m.k;

    if (m.q_mLmin > _cfg.qMaxOperativo_mLmin) {
        m.calidad |= FUERA_RANGO_OPERATIVO;
    }

    if (_cfg.calibracionDocumentada) {
        if (m.q_mLmin < _cfg.qMinCal_mLmin ||
            m.q_mLmin > _cfg.qMaxCal_mLmin) {
            m.calidad |= FUERA_RANGO_CAL;
        } else {
            m.calidad |= MEDICION_EN_RANGO_CAL;
        }
    }
}

// Se calcula desde el entero acumulado, sin sumar incrementos
// flotantes y sin depender del caudal mostrado.
m.volumenEstimado_L =
    static_cast<double>(m.pulsosMedicion) / (60.0 * m.k);

const bool nuevos =
    m.pulsosMedicion != _anteriorMedicion;
_anteriorMedicion = m.pulsosMedicion;

if (!seco && !bombaEmpuja &&
    (nuevos || isfinite(m.q_mLmin))) {
    m.calidad |= SIN_IMPULSION;
}

return m;

```

```

    return m;
}

double Caudalimetro::caudalMedio_mLmin(
    const Muestra& inicial, const Muestra& final) {

    const uint16_t iniciado = INICIADO;

    if (!(inicial.calidad & iniciado) ||
        !(final.calidad & iniciado) ||
        inicial.epoca != final.epoca ||
        (inicial.calidad & SECO) ||
        (final.calidad & SECO) ||
        !isfinite(inicial.k) ||
        inicial.k <= 0.0 ||
        inicial.k != final.k ||
        final.t_us <= inicial.t_us ||
        final.pulsosMedicion < inicial.pulsosMedicion) {
        return NAN;
    }

    const uint64_t dn =
        final.pulsosMedicion - inicial.pulsosMedicion;
    const int64_t dt = final.t_us - inicial.t_us;

    return static_cast<double>(dn) * 1.0e9 /
        (inicial.k * static_cast<double>(dt));
}

```

Convención de frontera: un pulso se atribuye al intervalo en que termina su validación. Eso hace reproducible el conteo, pero no elimina la cuantización ni garantiza que todo el volumen físico asociado al pulso haya pasado dentro de ese intervalo.

3.4. Archivo `registro_ensayos.h`

Cada fila guarda instantáneas completas, no referencias a objetos que puedan cambiar después. El módulo no inventa simultaneidad entre alimentación y permeado.

```

#pragma once

#include <Arduino.h>
#include <math.h>
#include <stddef.h>
#include <stdint.h>

#include "esp_timer.h"
#include "caudalimetro.h"
#include "config_metrologia.h"

class RegistroEnsayos {
public:
    enum class Tipo : uint8_t {
        INICIO = 0,
        PERIODICA = 1,
        FIN = 2
    };
};

```

```

enum Estado : uint16_t {
    CAMBIO_EPOCA = 1U << 0,
    OMISIONES = 1U << 1
};

struct Fila {
    uint16_t sesion = 0;
    Tipo tipo = Tipo::INICIO;
    uint16_t estado = 0;

    // Tiempo relativo al origen administrativo de la sesion.
    // Las marcas exactas de cada sensor siguen en a.t_us y p.t_us.
    int64_t tRegistro_us = 0;
    double rpm = 0.0;

    Caudalimetro::Muestra a;
    Caudalimetro::Muestra p;
};

bool activo() const { return _activo; }
bool incompleto() const { return _incompleto; }
size_t cantidad() const { return _n; }
uint64_t intervalosOmitidos() const { return _omitidos; }

const Fila* fila(size_t i) const {
    return i < _n ? &_filas[i] : nullptr;
}

// El objeto debe ser global o estatico, no una variable grande
// en la pila de una tarea.
bool iniciar(
    Caudalimetro& a, Caudalimetro& p,
    double rpm, bool bombaEmpuja) {

    if (_activo ||
        _sesion >= Metrologia::MAX_SESIONES ||
        _n + 2 > Metrologia::MAX_FILAS ||
        !isfinite(rpm) || rpm < 0.0) {
        return false;
    }

    const int64_t origen = esp_timer_get_time();

    const auto ma = a.capturar(bombaEmpuja);
    const auto mp = p.capturar(bombaEmpuja);

    if (!iniciadas(ma, mp)) return false;

    ++_sesion;
    _activo = true;
    _inicio_us = origen;
    _proximo_us = origen + Metrologia::INTERVALO_LOG_US;
    _epocaA = ma.epoca;
    _epocaP = mp.epoca;
    _estadoSesion = 0;

    // INICIO tiene tRegistro = 0 por definicion.

```

```

        // Los tiempos de captura reales no se modifican.
        guardar(Tipo::INICIO, 0, rpm, ma, mp);
        return true;
    }

    // Llamar frecuentemente desde la tarea propietaria.
    // No espera ni fabrica filas de instantes pasados.
    bool tick(
        Caudalimetro& a, Caudalimetro& p,
        double rpm, bool bombaEmpuja) {

        if (!_activo) return false;

        const int64_t ahora = esp_timer_get_time();
        if (ahora < _proximo_us) return false;

        const uint64_t vencidos =
            static_cast<uint64_t>(
                (ahora - _proximo_us) /
                Metrologia::INTERVALO_LOG_US) + 1;

        _proximo_us +=
            static_cast<int64_t>(vencidos) *
            Metrologia::INTERVALO_LOG_US;

        // Una fila se reserva siempre para FIN.
        if (_n >= Metrologia::MAX_FILAS - 1 ||
            !isfinite(rpm) || rpm < 0.0) {
            omitir(vencidos);
            return false;
        }

        const auto ma = a.capturar(bombaEmpuja);
        const auto mp = p.capturar(bombaEmpuja);

        if (!iniciadas(ma, mp)) {
            omitir(vencidos);
            return false;
        }

        if (vencidos > 1) omitir(vencidos - 1);

        guardar(
            Tipo::PERIODICA,
            esp_timer_get_time() - _inicio_us,
            rpm, ma, mp);

        return true;
    }

    bool finalizar(
        Caudalimetro& a, Caudalimetro& p,
        double rpm, bool bombaEmpuja) {

        if (!_activo || !isfinite(rpm) || rpm < 0.0) {
            return false;
        }
    }

```

```

// Captura final obligatoria: no usa la ultima fila periodica.
const auto ma = a.capturar(bombaEmpuja);
const auto mp = p.capturar(bombaEmpuja);

if (!iniciadas(ma, mp)) return false;

const int64_t ahora = esp_timer_get_time();

// Si habia periodicas vencidas no atendidas, se declaran.
// FIN es un evento distinto y no las reemplaza.
if (ahora >= _proximo_us) {
    const uint64_t vencidos =
        static_cast<uint64_t>(
            (ahora - _proximo_us) /
            Metrologia::INTERVALO_LOG_US) + 1;
    omitir(vencidos);
}

guardar(
    Tipo::FIN, ahora - _inicio_us,
    rpm, ma, mp);

_activo = false;
return true;
}

// No se borra una sesion mientras esta activa.
bool limpiar() {
    if (!_activo) return false;

    _n = 0;
    _sesion = 0;
    _omitidos = 0;
    _incompleto = false;
    _estadoSesion = 0;
    return true;
}

// Exportar solo fuera de ensayo evita que una salida lenta
// bloquee la tarea propietaria durante la adquisicion.
// Devuelve false si Print informa error de escritura.
bool exportarCSV(Print& salida) const {
    if (!_activo) return false;

    salida.clearWriteError();

    salida.println("# esquema=metrologia_nucleo_1");
    salida.println("# tiempo_sensor=us_desde_arranque");
    salida.println("# volumen=L_estimados_por_pulsos");
    salida.println("# nan=dato_no_disponible");
    salida.print("# incompleto=");
    salida.println(_incompleto ? 1 : 0);
    salida.print("# intervalos_omitidos=");
    imprimirU64(salida, _omitidos);
    salida.println();

    salida.println(

```

```

        "sesion,tipo,estado,t_reg_us,rpm,"
        "tA_us,epocaA,calidadA,kA,flancosA,aceptadosA,"
        "nA,rechazosA,edadA_us,fA_Hz,qA_mLmin,vA_L,"
        "tP_us,epocaP,calidadP,kP,flancosP,aceptadosP,"
        "nP,rechazosP,edadP_us,fP_Hz,qP_mLmin,vP_L");

for (size_t i = 0; i < _n; ++i) {
    const Fila& r = _filas[i];

    salida.print(r.sesion);
    salida.print(',');
    salida.print(static_cast<unsigned>(r.tipo));
    salida.print(',');
    salida.print(r.estado);
    salida.print(',');
    imprimirI64(salida, r.tRegistro_us);
    salida.print(',');
    imprimirReal(salida, r.rpm);
    salida.print(',');
    imprimirMuestra(salida, r.a);
    salida.print(',');
    imprimirMuestra(salida, r.p);
    salida.println();
}

return salida.getWriteError() == 0;
}

private:
    Fila _filas[Metrologia::MAX_FILAS] = {};

    size_t _n = 0;
    uint16_t _sesion = 0;
    bool _activo = false;
    bool _incompleto = false;

    uint64_t _omitidos = 0;
    int64_t _inicio_us = 0;
    int64_t _proximo_us = 0;

    uint32_t _epocaA = 0;
    uint32_t _epocaP = 0;
    uint16_t _estadoSesion = 0;

    static bool iniciadas(
        const Caudalimetro::Muestra& a,
        const Caudalimetro::Muestra& p) {

        return (a.calidad & Caudalimetro::INICIADO) &&
            (p.calidad & Caudalimetro::INICIADO);
    }

    void omitir(uint64_t cantidad) {
        if (cantidad == 0) return;
        _omitidos += cantidad;
        _incompleto = true;
        _estadoSesion |= OMISIONES;
    }

```

```

}

void guardar(
    Tipo tipo, int64_t relativo,
    double rpm,
    const Caudalimetro::Muestra& a,
    const Caudalimetro::Muestra& p) {

    // La API reserva FIN y evita llamar sin espacio.
    if (_n >= Metrologia::MAX_FILAS) {
        _incompleto = true;
        return;
    }

    if (a.epoca != _epocaA || p.epoca != _epocaP) {
        _estadoSesion |= CAMBIO_EPOCA;
        _incompleto = true;
    }

    Fila& r = _filas[_n++];
    r.sesion = _sesion;
    r.tipo = tipo;
    r.estado = _estadoSesion;
    r.tRegistro_us = relativo;
    r.rpm = rpm;
    r.a = a;
    r.p = p;
}

static void imprimirU64(Print& s, uint64_t valor) {
    char texto[24];
    snprintf(
        texto, sizeof(texto), "%llu",
        static_cast<unsigned long long>(valor));
    s.print(texto);
}

static void imprimirI64(Print& s, int64_t valor) {
    char texto[24];
    snprintf(
        texto, sizeof(texto), "%lld",
        static_cast<long long>(valor));
    s.print(texto);
}

static void imprimirReal(Print& s, double valor) {
    if (isfinite(valor)) s.print(valor, 9);
    else s.print("nan");
}

static void imprimirMuestra(
    Print& s, const Caudalimetro::Muestra& m) {

    imprimirI64(s, m.t_us);
    s.print(',');
    s.print(m.epoca);
    s.print(',');
    s.print(m.calidad);
}

```

```

        s.print(m.caudal);
        s.print(',');
        imprimirReal(s, m.k);
        s.print(',');
        imprimirU64(s, m.flancos);
        s.print(',');
        imprimirU64(s, m.aceptados);
        s.print(',');
        imprimirU64(s, m.pulsosMedicion);
        s.print(',');
        imprimirU64(s, m.rechazados);
        s.print(',');
        imprimirI64(s, m.edadPulso_us);
        s.print(',');
        imprimirReal(s, m.frecuencia_Hz);
        s.print(',');
        imprimirReal(s, m.q_mLmin);
        s.print(',');
        imprimirReal(s, m.volumenEstimado_L);
    }
};

```

3.5. Archivo `banco_metrologia.ino`

Este programa permite comprobar inicialización, modos, sesiones y CSV. **No acciona el motor ni sustituye el programa de control de la planta.** Mantiene la salida de habilitación en el estado de parada declarado por ustedes; esa polaridad debe verificarse físicamente.

El servicio de interrupciones GPIO se instala aquí una sola vez. Si el firmware integral ya lo instala, debe existir un único responsable de esa inicialización y deben revisarse sus opciones.

```

#include <Arduino.h>

#include "driver/gpio.h"
#include "esp_timer.h"

#include "config_metrologia.h"
#include "caudalimetro.h"
#include "registro_ensayos.h"

static Caudalimetro alimentacion(Metrologia::SENSOR_A);
static Caudalimetro permeado(Metrologia::SENSOR_P);
static RegistroEnsayos registro;

static bool listo = false;

static void indicarResultado(const char* operacion, bool ok) {
    Serial.print(operacion);
    Serial.println(ok ? ": OK" : ": NO REALIZADO");
}

void setup() {
    // Preparar los niveles antes de configurar las salidas reduce
    // transiciones de arranque por software. No garantiza el estado
    // electrico previo a setup ni durante un reinicio.
    digitalWrite(Metrologia::PIN_ENA, HIGH);
}

```

```

pinMode(Metrologia::PIN_ENA, OUTPUT);

digitalWrite(Metrologia::PIN_PUL, LOW);
pinMode(Metrologia::PIN_PUL, OUTPUT);

digitalWrite(Metrologia::PIN_DIR, LOW);
pinMode(Metrologia::PIN_DIR, OUTPUT);

Serial.begin(115200);

// Servicio ordinario: no se afirma funcionamiento cache-safe.
const esp_err_t err = gpio_install_isr_service(0);
if (err != ESP_OK) {
    Serial.print("Fallo instalando servicio GPIO: ");
    Serial.println(static_cast<int>(err));
    return;
}

const bool okA = alimentacion.begin();
const bool okP = permeado.begin();

if (!okA || !okP) {
    Serial.println("Fallo inicializando sensores; motor
detenido.");
    return;
}

// Estado inicial de diagnostico, sin totalizar volumen.
alimentacion.setModoSeco(true);
permeado.setModoSeco(true);

listo = true;

Serial.println("Banco de comprobacion metrologica.");
Serial.println("No genera pasos de motor.");
Serial.println("d: modo seco, fuera de sesion");
Serial.println("h: modo medicion, fuera de sesion");
Serial.println("i: iniciar sesion");
Serial.println("f: finalizar sesion");
Serial.println("e: exportar CSV, fuera de sesion");
Serial.println("l: limpiar registro, fuera de sesion");
Serial.println("a: limpiar alarmas de seco, fuera de sesion");
}

void loop() {
    if (!listo) {
        delay(10);
        return;
    }

    // En este programa el motor permanece detenido.
    // En la planta, estos dos valores deben proceder del controlador
    // real, no de un indicador de interfaz.
    constexpr bool bombaEmpuja = false;
    constexpr double rpm = 0.0;

    registro.tick(alimentacion, permeado, rpm, bombaEmpuja);

```

```

if (Serial.available()) {
    const char comando = static_cast<char>(Serial.read());

    switch (comando) {
    case 'd':
    case 'h':
        if (registro.activo()) {
            Serial.println("Finalice la sesion antes de cambiar
modo.");
        } else {
            const bool seco = comando == 'd';
            alimentacion.setModoSeco(seco);
            permeado.setModoSeco(seco);
            Serial.println(seco ? "Modo seco." : "Modo medicion.");
        }
        break;

    case 'i':
        indicarResultado(
            "Inicio",
            registro.iniciar(
                alimentacion, permeado, rpm, bombaEmpuja));
        break;

    case 'f':
        indicarResultado(
            "Fin",
            registro.finalizar(
                alimentacion, permeado, rpm, bombaEmpuja));
        break;

    case 'e':
        indicarResultado("CSV", registro.exportarCSV(Serial));
        break;

    case 'l':
        indicarResultado("Limpieza", registro.limpiar());
        break;

    case 'a':
        if (!registro.activo()) {
            alimentacion.limpiarAlarmaSeco();
            permeado.limpiarAlarmaSeco();
            Serial.println("Alarmas limpiadas.");
        } else {
            Serial.println("No se borran alarmas durante la
sesion.");
        }
        break;

    default:
        break;
    }
}

delay(1);
}

```

4. Integración que debe respetar el firmware de la planta

El programa anterior resuelve el **núcleo de adquisición y registro**, no las funciones de seguridad que no fueron entregadas. Para integrarlo sin reintroducir inconsistencias:

Una sola tarea propietaria debe modificar sensores y registro. Las órdenes web llegan por cola; el servidor no modifica directamente esos objetos.

El SCADA recibe copias de las muestras. No lee objetos mientras otra tarea los está modificando.

El registro conserva NAN como ausencia de dato. En JSON se transforma en `null`, acompañado de sus marcas de calidad; nunca se envía `NaN` como número JSON.

La parada del motor no cierra necesariamente la adquisición. Si interesa medir flujo residual, la sesión debe terminar después de un intervalo de observación definido.

Las escrituras de flash, OTA y persistencia no se realizan durante adquisición con esta implementación ordinaria de ISR. Si se necesitan, debe verificarse una cadena IRAM completa o una captura hardware apropiada.

La configuración queda fija durante la sesión. Un cambio de modo queda detectado como cambio de época; no debe interpretarse el intervalo completo como una medición homogénea.

La finalización del log no condiciona una parada de seguridad. Primero se ejecuta la acción de seguridad y después se intenta registrar su resultado.

4.1. Balance hidráulico

El balance con signo debe conservarse, pero **no siempre representa directamente el caudal instantáneo de retorno.**

La conservación general incluye almacenamiento:

$$Q_{\mathrm{alim}} - Q_{\mathrm{perm}} - Q_{\mathrm{ret}}$$

$$\frac{dV_{\mathrm{circuito}}}{dt}$$

En régimen aproximadamente estacionario, o sobre intervalos donde el almacenamiento sea despreciable:

$$\overline{Q_{\mathrm{ret}}} \approx \overline{Q_{\mathrm{alim}}} - \overline{Q_{\mathrm{perm}}}$$

Para la tesis recomiendo calcular el balance sobre **incrementos de volumen de intervalos comparables**, conservando las marcas temporales individuales y evaluando el efecto de su desfase.

El rendimiento debe quedar **no disponible** si el denominador no permite una estimación defendible:

$$Y=100,\frac{Q_{\mathrm{perm}}}{Q_{\mathrm{alim}}}$$

No debe sustituirse un denominador pequeño por una constante para obtener un porcentaje aparentemente válido.

4.2. Darcy y presión

Hasta disponer de presión medida, **TMP y Darcy deben estar ausentes o inequívocamente marcados como inválidos**. `TMP = 0.00` solo es aceptable si todos los consumidores respetan `valido = false` ; de lo contrario comunica una presión medida inexistente.

La misma regla se aplica a un caudal recíproco sin período: ausencia de señal no demuestra necesariamente caudal físico nulo.

5. Pruebas necesarias para levantar el rechazo

La aprobación no se obtiene por adoptar mi implementación, sino por demostrar **requisitos verificables**. La versión propuesta también debe someterse a estas pruebas.

5.1. Verificación del software

Deben conservarse compilación reproducible, versiones exactas, configuración y resultados de prueba.

Prueba	Estímulo	Criterio
Conteo	Tren conocido de pulsos admisibles	Coincidencia con el generador tras el transitorio inicial definido
Rechazo	Pulsos bajo cada umbral	Rechazo conforme a la máquina de estados
Ventanas	Pulsos cruzando capturas	Sin pérdida por reinicio de ventana
Timeout	Pausa mayor al límite	Caudal no disponible; recuperación con períodos nuevos
Modo seco	Señal durante seco y transición	Sin incremento volumétrico en seco ni período heredado
Sesiones	Inicio repetido, final y reinicio	Sin sesiones superpuestas ni filas previas reutilizadas
Capacidad	Agotar filas y sesiones	Sin sobrescritura; <code>FIN</code> reservado y omisiones visibles
Carga	Motor, Wi-Fi y clientes simultáneos	Sin pérdidas dentro del criterio metrológico establecido

El arranque necesita atención especial: la validación de un nivel HIGH completo puede descartar el primer pulso. **Debe medirse y documentarse**, no esconderse modificando el contador esperado.

5.2. Caracterización metrológica

Para cada sensor se requiere un procedimiento que incluya **rango, repetibilidad y presupuesto**

Para cada sensor se requiere un procedimiento que incluya **rango, repetibilidad y presupuesto de incertidumbre**.

Patrón gravimétrico: calibración o verificación de la balanza, resolución, tara, deriva, evaporación y densidad del agua a la temperatura medida.

Rango real: puntos ascendentes y descendentes, umbral de arranque, histéresis y ausencia de respuesta a bajo caudal.

Modelo de conversión: comprobar si basta un único K o si existe dependencia con caudal, montaje o pulsación.

Comparación de conteo: pulsos registrados frente a volumen de referencia, incluyendo cuantización y fronteras temporales.

Interferencia: ensayos secos y húmedos con motor y comunicaciones activos.

Criterio previo de aceptación: error e incertidumbre máximos compatibles con los objetivos de la tesis.

La resolución de **0.1 g no equivale por sí sola a incertidumbre de 0.1 g**. Tampoco una probeta fina convierte automáticamente el procedimiento en una calibración trazable.

5.3. Seguridad del banco

Antes de ensayos con la membrana deben documentarse **la protección de presión y la parada efectiva**, incluyendo su comportamiento ante pérdida de Wi-Fi, bloqueo de tareas y reinicio del ESP32.

El rechazo puede levantarse para ensayos de caracterización cuando se verifiquen el núcleo corregido, su integración y la protección independiente del banco. La aprobación de producción exige además la calibración aplicable, las pruebas bajo carga y la revisión del firmware completo. Ninguna bandera en C++ sustituye esas evidencias.